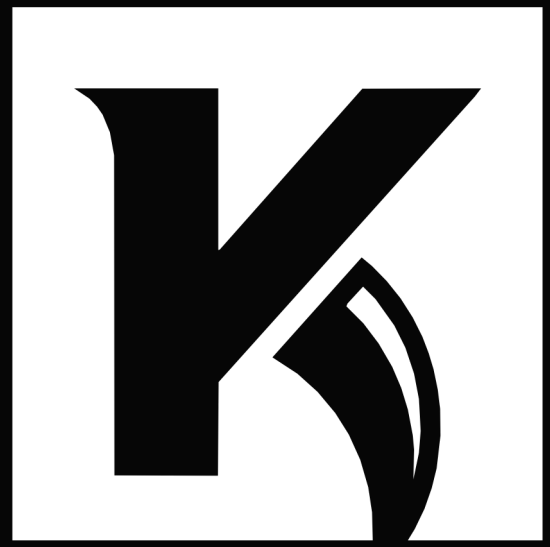
KUJO

Trust at the speed of AI.

Kujo makes agent-driven engineering work inspectable, repeatable and accountable across existing workflows.



Software production is outpacing verification.

Agents can report completion. Teams still need a reviewable chain from requested work to release decision.

01 **The claim**

“Tests passed. Ready to merge.” An illustrative agent statement.

02 **The evidence**

Command, exit code, changed revision, environment and retained failure output.

03 **The gap**

Useful logs already exist. Connecting them across tools and review boundaries remains work.

Agents now work across the repository.

GitHub documents agents that plan changes, edit code and review pull requests. More delegated work makes the execution record part of the review.

01

More scope

Agents move from suggestions to repository tasks.

02

More handoffs

The reviewer needs the revision, executed checks and retained failures.

03

Why now

Existing platforms establish the workflow; Kujo proposes a portable evidence layer.

A passing summary can hide a failed checkout.

A coupon changes a $50 cart to $50.99 instead of $45. The reviewer needs the failing test, exact change and unchanged checks.

01 **Reconstruct**

Find the task, revision, environment and actual command result.

02 **Correct**

Fix shipping eligibility without weakening the test.

03 **Review**

Compare before and after evidence. Measure buyer investigation time in pilots.

Build with clarity. Operate with control.

The language is the long-term foundation. Existing tools offer an earlier route into engineering workflows.

01 **Clarity**

Kujo language / Spec / Eval: explicit intent and testable outcomes.

02 **Context**

MCP / CaseFile / RunLedger: useful inputs, debugging evidence and run records.

03 **Control**

Dispatch / Workcell / ShipCheck: orchestration, bounded execution and release checks.

Existing repositories + coding agents

explicit component contracts

Kujo runtime + CLI

Accountability spans the whole task.

Each component owns a specific responsibility. A reviewable workflow composes their artifacts explicitly.

01 **Instruction → Context**

Spec / PackWrite → MCP and repository context

→ 02 **Execution → Evaluation**

Dispatch / Workcell → Eval result artifacts

→ 03 **Evidence → Review → Release**

CaseFile / RunLedger → reviewer → ShipCheck gate

Human authority remains explicit at the release decision.

A working foundation precedes this raise.

The language, execution tools and integration adapters are inspectable in public source.

01 **Language and VM**

Kujo v1.9.0 release documentation; compiler and VM source pinned for diligence.

02 **Workflow tooling**

CaseFile captures commands. Eval runs checks. Workcell documents bounded execution.

03 **Existing entry points**

Command Code, Hermes, OpenCode and Pi adapters have documented releases and scoped verification.

Keep the failure. Verify the correction.

A reproducible checkout regression: a coupon must preserve free shipping. The original tests fail, a one-line correction passes them.

01 **Before**

Python test fails: 5099 cents returned instead of 4500.

02 **Correction**

Check the original subtotal for shipping eligibility.

03 **After**

The same three tests pass. CaseFile retains both runs; Eval reruns the gate.

Adopt the ecosystem before the language.

Keep application code in its existing language. Introduce accountability at the workflow boundary.

01

Today: wrap the task

Use CaseFile for command evidence and Eval for explicit repository checks.

02

Then: control execution

Adopt Dispatch or Workcell after validating the target environment and runtime pins.

03

Over time: go deeper

Choose Kujo for new agent tooling where native language integration adds value.

Start with the platform engineering lead.

Initial buyer hypothesis: a team operating coding agents across repositories and reconstructing failures during review.

01 **Purchase trigger**

An agent regression crosses a review boundary and the team must reconstruct what ran.

02 **What we know**

Local evidence tools work; integration adapters and a public catalog are available.

03 **What we are testing**

Whether shared evidence saves review effort and earns a recurring team budget.

Open tools today. A team service to validate.

Primary paid wedge: shared agent-run evidence and review for engineering teams. Application code stays in its existing language.

01

Available now · open source

Local CLI tools and documented host integrations; no Kujo subscription required.

02

Planned · team service

Proposed $500/month: evidence intake, retention, access controls, review and export.

03

Planned · enterprise

Separate $24K/year hypothesis for deployment, policy and support. Excluded from the base case.

One team subscription. Explicit revenue milestones.

Base pricing hypothesis: $500 per team per month. Customer counts below are scenarios, not forecasts or achieved revenue.

17 teams

$102K ARR · first repeatable revenue milestone

167 teams

$1.002M ARR · requires retained paid cohorts

1,000 teams

$6M ARR · long-term sensitivity, not a 24-month promise

Portable evidence across existing workflows.

GitHub integrates agents and review. LangGraph persists workflow state. Kujo composes local execution artifacts with explicit evaluation.

01 **Demonstrated**

Before/after command bundles, unchanged checks and an inspectable patch in the checkout demo.

02 **Interoperable**

Existing application languages and documented host adapters; exports stay reviewable outside a chat.

03 **To earn**

A reliable shared service, useful integrations and retained buyer usage. Competitive parity is not a benchmark.

Turn a first evidence run into a retained team.

Integration-led distribution: one useful repository workflow, then a buyer conversation about shared review.

01

Discover → activate

Docs, MCP catalog and host adapters → first captured failure and passing rerun.

02

Validate → pilot

Interview 12 buyers; qualify design partners against an actual incident and budget owner.

03

Convert → retain

Offer a scoped paid pilot after readiness checks; measure weekly use, review time and renewal.

Public engineering and distribution are visible.

Technical momentum is independently inspectable. Commercial validation is the next milestone.

01 **Engineering**

Kujo runtime and tools have pinned public source; this deck includes a rerun checkout regression.

02 **Distribution**

Kujolang.ai MCP is listed in the GitHub MCP Registry, observed November 11, 2026.

03 **Commercial stage**

No verified paying customers or revenue in the reviewed evidence. Pilot and retention validation remain ahead.

A founder with a public shipping record.

Robert DeVore builds commercial web software and the Kujo language ecosystem.

01 **Independent shipping**

Public biography describes more than 20 years building software; tenure is self-reported.

02 **Third-party adoption**

WebDevStudios reported adopting his plugins in July 2025. No unsupported plugin count is used.

03 **Team to build**

Stage execution reliability, product design, developer experience and customer development hires.

Fund commercialization with staged commitments.

Proposed $6M raise: a 24-month operating plan with $900K reserve. Release hiring against validation, paid pilots and retention.

$6M

proposed raise · founder confirmation pending

$30M

proposed post-money SAFE cap · negotiable

24 mo

modeled spend horizon · no revenue offsets

0–6 mo
Flagship evidence workflow

6–12 mo
External validation + pilots

12–24 mo
Paid retention + repeatability

Discuss the raise. Inspect the evidence.

Proposed $6M financing conversation. Start with a technical diligence walkthrough and the team-service thesis.

KUJO

Trust at the speed of AI.

Investor conversations ↗